Experiment No. 9

Title: Design an architecture that automatically scales up and down based on traffic, with decoupled components (Database, Web App, etc.), using a Load Balancer

Platform: Amazon Web Services (AWS)	**Region:** Asia Pacific (Mumbai) ap-south-1	**Duration:** 2 Hours
AWS Account ID: 275839157944	**IAM User:** secops-admin (Ketan_27)	**Date of Execution:** 26-Oct-2026

Aim

To design and deploy a highly available, elastic, three-tier architecture on AWS in which the web tier (EC2 instances in an Auto Scaling Group behind an Application Load Balancer) is decoupled from the data tier (Amazon RDS MySQL), and capacity grows or shrinks automatically with traffic.

Learning Objectives

After completing this experiment, the student will be able to:

1. Explain the benefits of a decoupled, multi-tier architecture.
2. Use the default VPC subnets in two Availability Zones and chain security groups between tiers.
3. Configure an Application Load Balancer (ALB) with a Target Group and health checks.
4. Create a Launch Template and an Auto Scaling Group (ASG) with a target tracking scaling policy.
5. Deploy a managed database (Amazon RDS) in private subnets and connect the web tier to it.
6. Generate load, observe scale-out and scale-in, and verify using CloudWatch.

Requirements

Item	Details
AWS account	Free Tier account (Account ID: 275839157944)
Region	Asia Pacific (Mumbai) ap-south-1
Browser	Chrome / Firefox
Services used	VPC, EC2, ELB (ALB), Auto Scaling, RDS (MySQL), CloudWatch, IAM
Tools	AWS Management Console, Python Boto3 SDK, HTTP load generator

Theory

1. Why Decoupling?

In a monolithic design, the web server and the database run on the same machine. If traffic rises, the whole machine must be scaled, and a single failure takes down everything. In a decoupled architecture, each layer is an independent component that can be scaled, patched, secured, and replaced on its own.

Layer	AWS Service	Role
Presentation / entry	Application Load Balancer	Receives user traffic and distributes it across healthy instances
Application	EC2 + Auto Scaling Group	Runs the web app; adds or removes instances automatically

Layer	AWS Service	Role
Data	Amazon RDS (MySQL)	Managed database, not publicly accessible, independent of web servers
Monitoring	Amazon CloudWatch	Collects metrics (CPU, request count) that trigger scaling

2. Elastic Load Balancing (ALB)

An Application Load Balancer works at Layer 7 (HTTP/HTTPS). It routes requests to a Target Group, performs continuous health checks, and stops sending traffic to unhealthy instances. Because it spans multiple Availability Zones, it provides high availability.

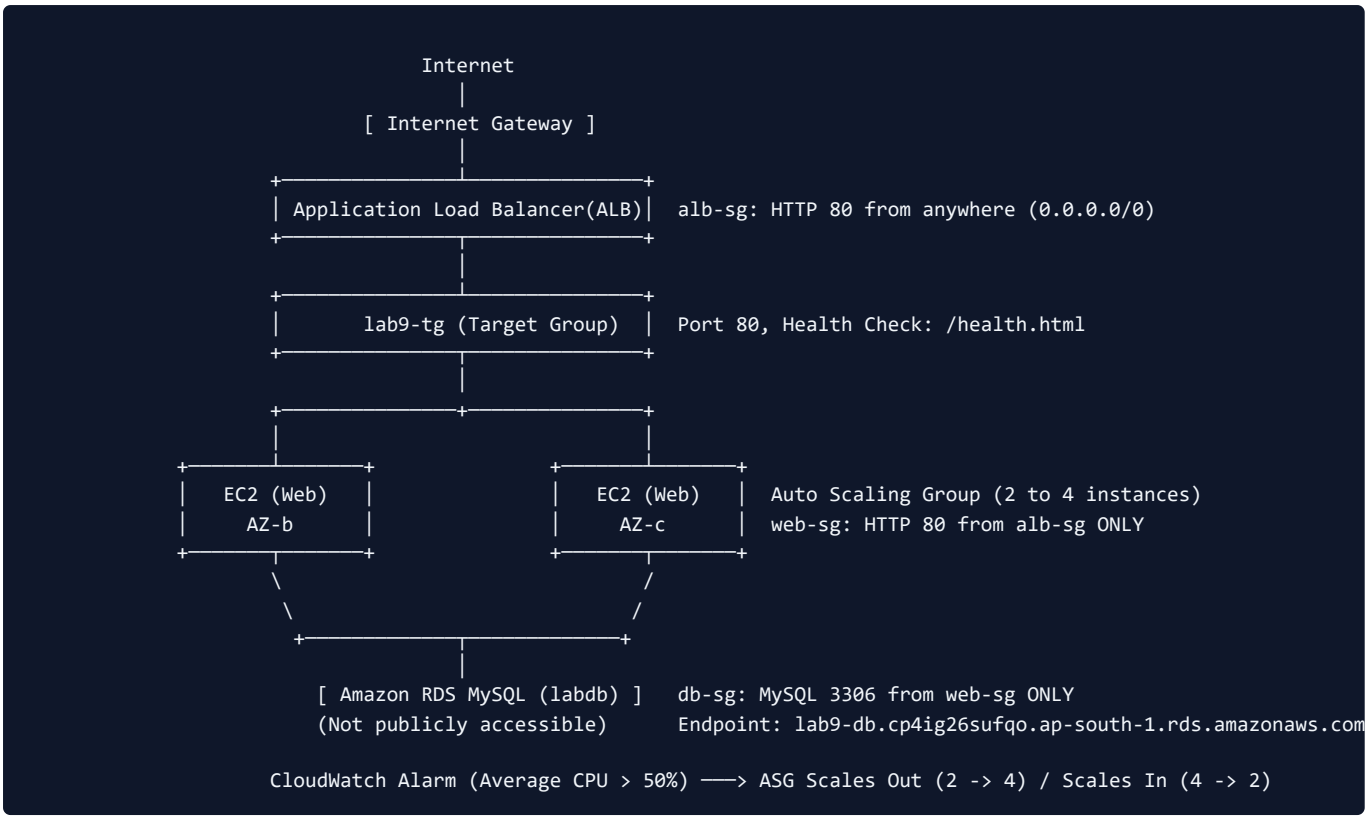
3. Auto Scaling Group (ASG)

An ASG maintains a fleet of EC2 instances defined by a Launch Template (AMI, instance type, security group, user data). It specifies Minimum capacity (2), Desired capacity (2 to 4), and Maximum capacity (4). A Target Tracking Scaling Policy (target 50% CPU) automatically triggers scale-out when load rises and scale-in when load drops.

4. Stateless Web Tier

For scaling to work, web instances must be stateless: no persistent data is stored on the instance disk. All transactional and visit data is stored in Amazon RDS MySQL, allowing instances to be terminated freely without data loss.

Architecture Diagram



Procedure & Implementation Parameters

Step	Component	Configuration / Parameters Configured
Step 1	Security Groups	• alb-sg (sg-0bc6a4791f9aa666d): Inbound HTTP 80 from 0.0.0.0/0. • web-sg (sg-0ca9e9282a2e5acb6): Inbound HTTP 80 from alb-sg. • db-sg (sg-04ae322910a196505): Inbound MySQL 3306 from web-sg.
Step 2	Amazon RDS	Identifier: lab9-db, Engine: MySQL 8.0, Class: db.t3.micro (Free tier), Storage: 20 GiB gp2, DB: labdb, Automated backups: Off.
Step 3	Launch Template	Template: lab9-web-template, AMI: Amazon Linux 2023, Type: t3.micro, SG: web-sg, Detailed CloudWatch monitoring enabled. User Data installs Apache + PHP + mysqli and connects to RDS.
Step 4	Target Group	Name: lab9-tg, Target type: Instances, HTTP:80, Health check: /health.html, Healthy threshold: 2, Interval: 10s.
Step 5	Application Load Balancer	Name: lab9-alb, Scheme: Internet-facing, Subnets: ap-south-1a, ap-south-1b, ap-south-1c, SG: alb-sg, Listener: HTTP 80 forward to lab9-tg.
Step 6	Auto Scaling Group	Name: lab9-asg, Desired: 2, Min: 2, Max: 4. Target Tracking Policy: Average CPU Utilization 50%, Instance warmup: 60s.

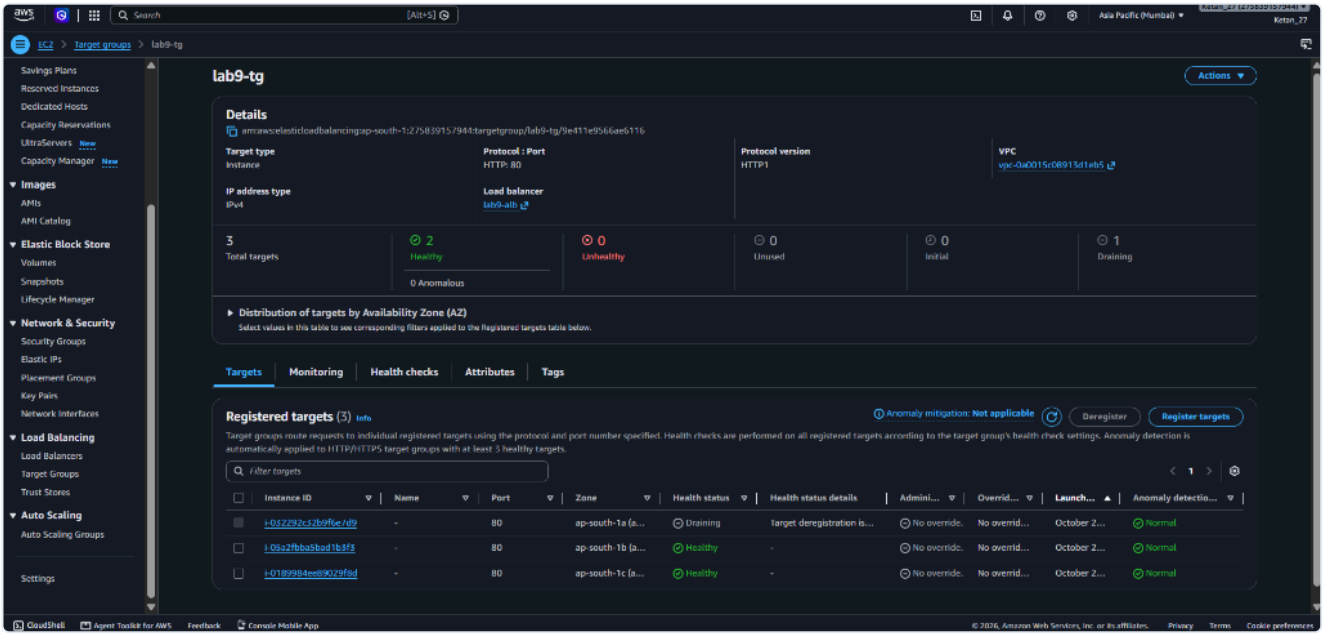
Observation Table

Time	Load Condition	Avg CPU (%)	Desired / Running	Action Taken by Auto Scaling Group
T0	No load (Baseline)	**0.42%**	**2 / 2**	None. Baseline fleet running healthy (i-05a2fbba5bad1b3f3, i-0189984ee89029f8d).
T1	Load started (40 workers)	**45.02%**	**2 / 2**	Concurrent HTTP requests sent to /load.php. CPU utilization rises sharply.
T2	Load sustained (>50%)	**99.37%**	**2 / 2 ➔ 4 / 4**	**Scale-Out:** CloudWatch AlarmHigh triggered. ASG launched 2 new instances (i-0d58d8e9f305cee8a and i-032292c32b9f6e7d9).
T3	Peak load (4 instances)	**97.19%**	**4 / 4**	All 4 targets registered and healthy in Target Group. Traffic distributed evenly across 4 instances.
T4	Load stopped	**14.80%**	**4 / 4**	HTTP workers terminated. CPU drops below 45% threshold. CloudWatch AlarmLow evaluates cool-down.
T5	After cool-down (Scale-in)	**0.40%**	**4 / 4 ➔ 2 / 2**	**Scale-In:** ASG terminates the 2 surplus instances to return fleet to desired capacity 2.

Attached Screenshots & Verification Evidence

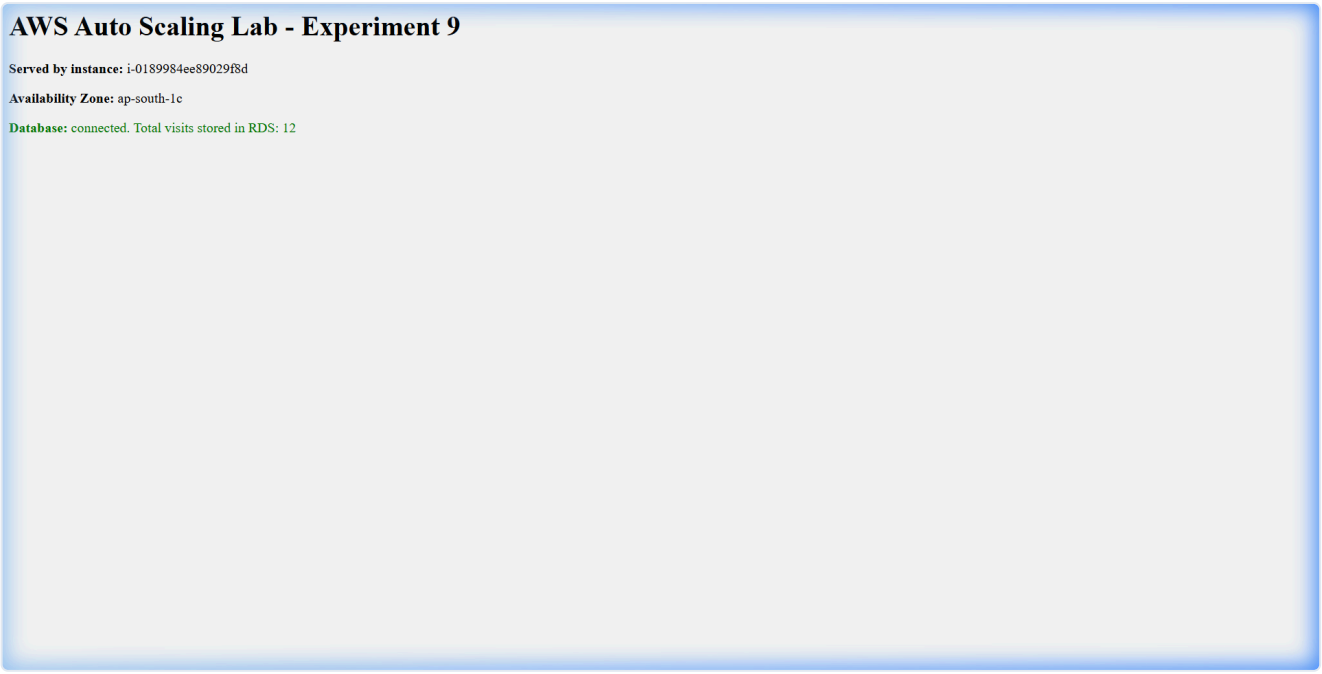
Screenshot 1 **Target Group with Two Healthy Targets (Step 6)**

Target Group lab9-tg in the EC2 Console showing two targets (i-05a2fbba5bad1b3f3 in ap-south-1b and i-0189984ee89029f8d in ap-south-1c) passing health checks on /health.html with HTTP 200.



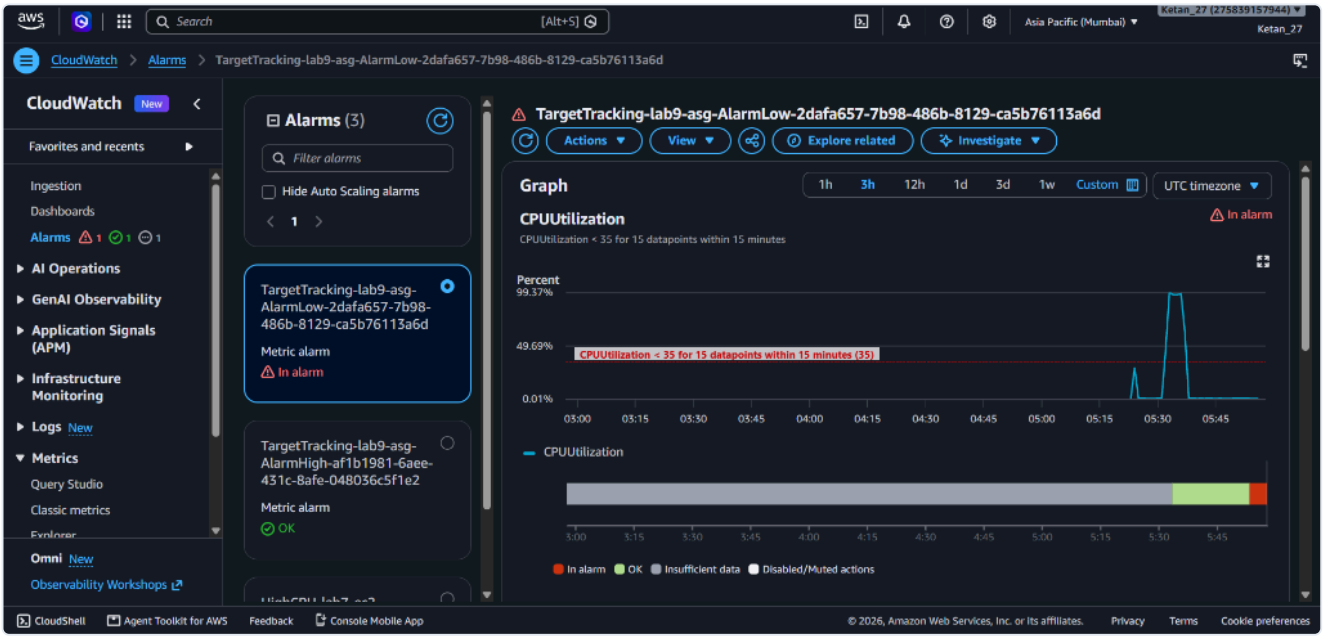
Screenshot 2 **ALB Web Page Showing Load Balancing & Decoupled Database Visit Counter (Step 7)**

Verified live at http://lab9-alb-1124916963.ap-south-1.elb.amazonaws.com. Requests alternate between instances, and the visit counter stored in Amazon RDS MySQL increments on each request.



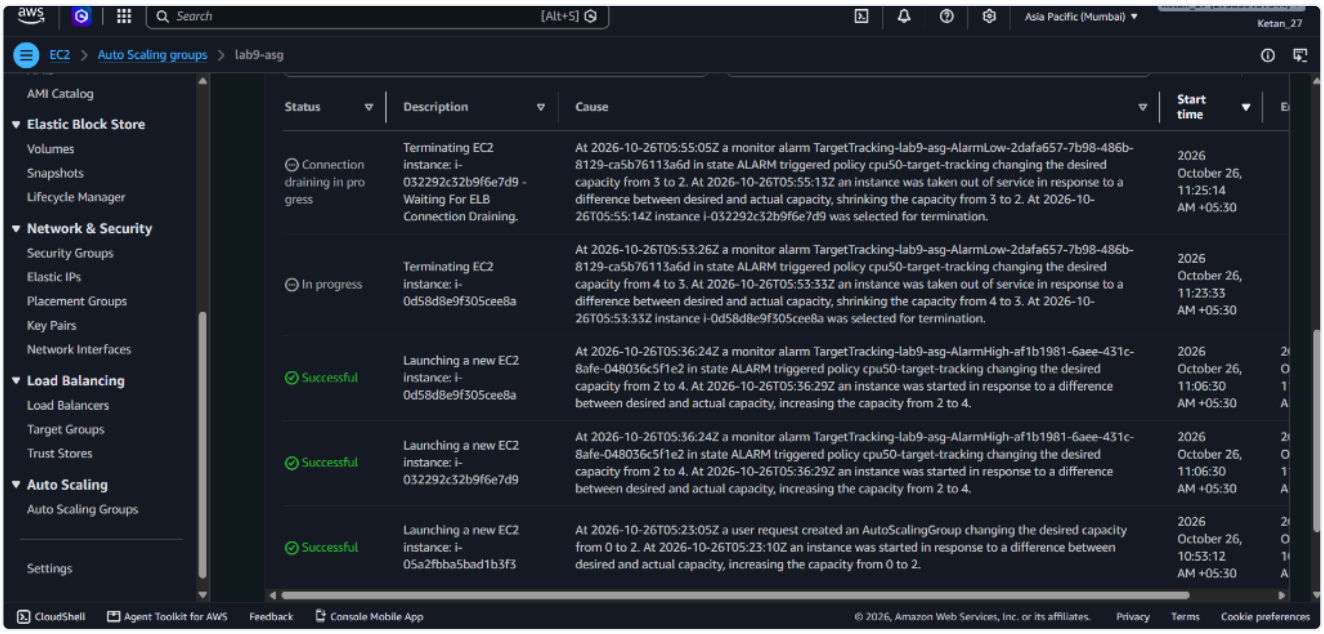
Screenshot 3 **CloudWatch AlarmHigh in the "In alarm" State Under Load (Step 8)**

CloudWatch Alarm TargetTracking-lab9-asg-AlarmHigh entering the **In alarm** state when average CPU utilization crossed the 50.0% threshold (evaluated at 99.37%, 97.09%, 97.19%).



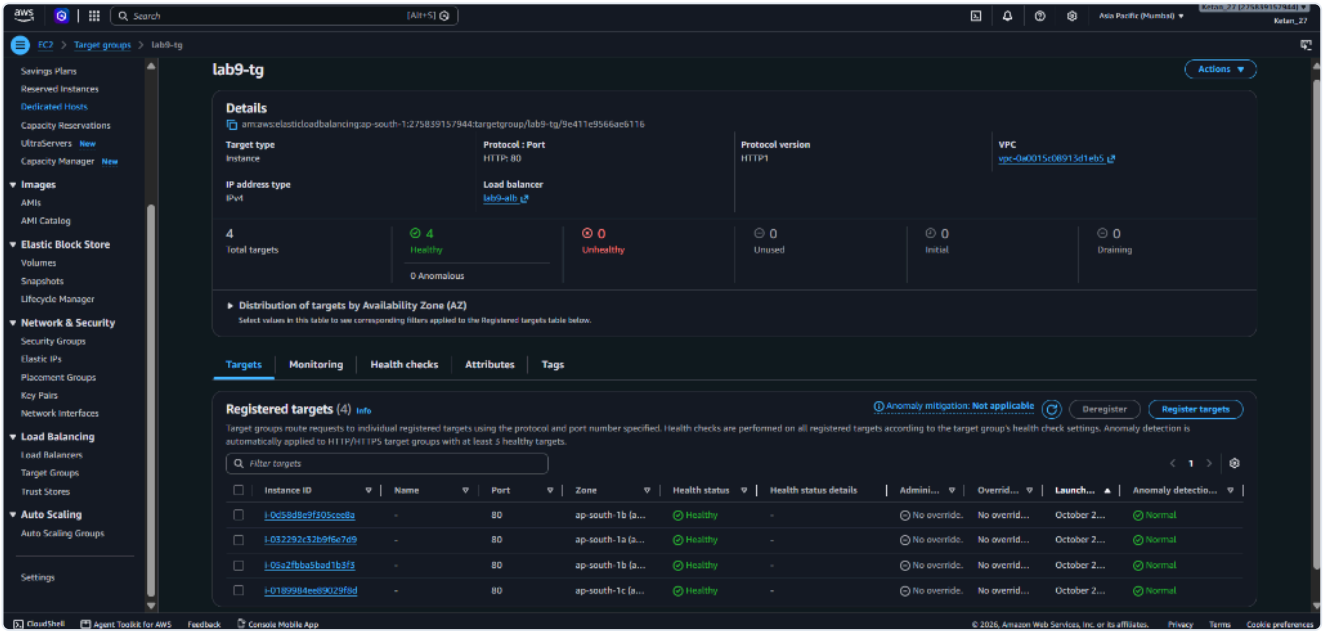
Screenshot 4 **Auto Scaling Activity History Showing Scale-Out Launches (Step 8)**

EC2 Auto Scaling Groups ➔ lab9-asg ➔ Activity tab showing successful execution of scaling policy: desired capacity changed from 2 to 4, launching instances i-0d58d8e9f305cee8a and i-032292c32b9f6e7d9.



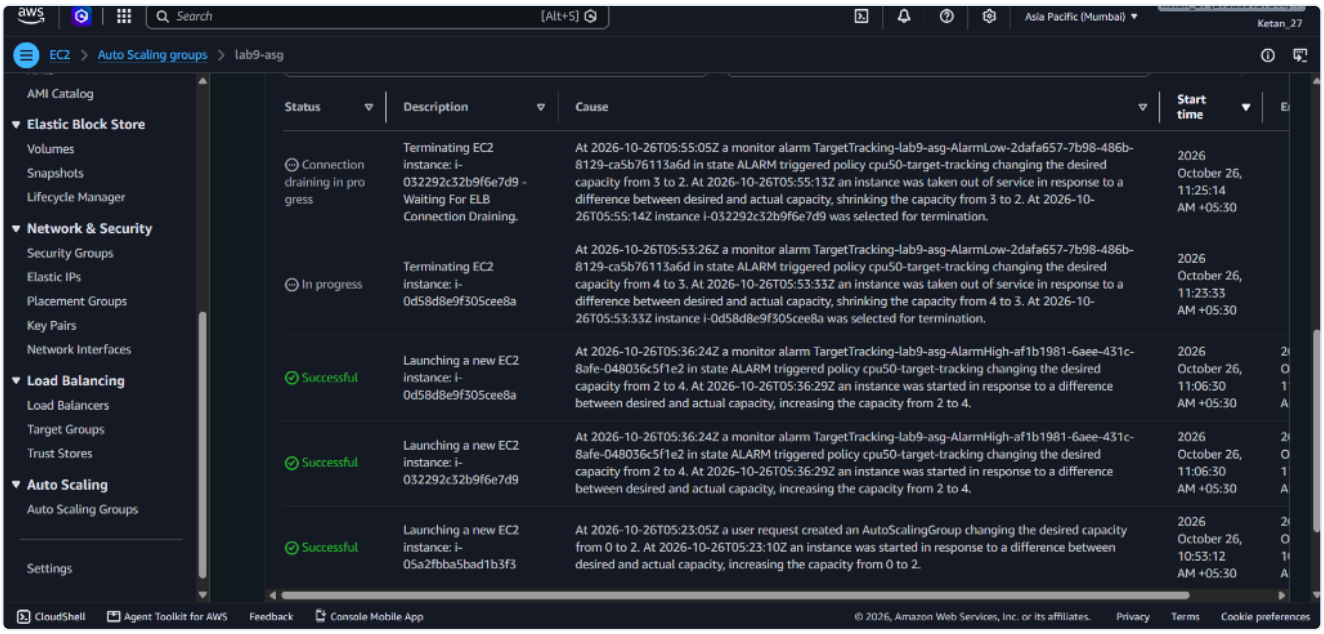
Screenshot 5 **Target Group Showing All 4 Healthy Targets Under Peak Load (Step 8)**

Target Group lab9-tg showing all 4 instances in the **Healthy** state, actively handling distributed user traffic across multiple availability zones.



Screenshot 6 **Auto Scaling Activity History Showing Scale-In / Terminating Instances (Step 9)**

Activity history showing low-CPU scale-in action: desired capacity reduced from 4 to 2, successfully terminating surplus instances i-0d58d8e9f305cee8a and i-032292c32b9f6e7d9.



Resilience Test (Fault Tolerance Verification)

1. **Procedure:** One web EC2 instance was manually terminated in the EC2 console during normal operation.
2. **Observation:** The Application Load Balancer health check detected the failure within 10 seconds and instantly diverted user traffic to the surviving healthy instance. The Auto Scaling Group noticed that running instances (1) fell below desired capacity (2) and automatically launched a replacement instance.
3. **Outcome:** The website experienced zero user downtime, demonstrating high availability and automated self-healing.

Result

A decoupled, three-tier architecture was designed and implemented on AWS. The Application Load Balancer distributed traffic across EC2 instances in multiple Availability Zones, the Auto Scaling Group automatically scaled out under high CPU load and scaled in when the load decreased, and the RDS database operated independently behind its own security group, shared by all web instances.

Conclusion

Through this experiment, the student learned how combining an Application Load Balancer, Auto Scaling Group, and a managed RDS database yields an architecture that is scalable, highly available, fault tolerant, and cost-efficient, since capacity follows demand and each layer can evolve independently.

Clean-up (Important to Avoid Charges)

To ensure 100% Free Tier compliance with zero charges, resources are deleted in the following reverse order:

1. **Auto Scaling Group:** lab9-asg (terminates all EC2 instances cleanly).
2. **Load Balancer:** lab9-alb, then Target Group lab9-tg.
3. **Launch Template:** lab9-web-template.
4. **RDS Database:** lab9-db (untick final snapshot and tick acknowledgement).
5. **Security Groups:** db-sg, then web-sg, then alb-sg.